

RaftScope: A Causally-Instrumented Raft Implementation with Space-Time Visualization

CSCI 546, Distributed Systems, Spring 2026

Karthik Venugopal Akshay Shivashankar Bharadwaj
Adithya Sudhindra
University of Southern California

1 Introduction

This report describes **RaftScope**, our final project for CSCI 546. The system has two components that together enable principled debugging of distributed consensus: (1) a complete C++ implementation of the Raft consensus algorithm built on the course skeleton using gRPC, and (2) a self-contained, browser-based space-time diagram visualizer that reconstructs the causal history of a run from per-node JSONL event logs.

The core thesis is that Raft correctness bugs (split votes, log divergence, liveness stalls) are most naturally diagnosed by inspecting the *causal* ordering of messages across nodes, not by correlating wall-clock timestamps from separate log files. RaftScope embeds Lamport timestamps in every RPC so the visualizer can draw causal arrows between events even when system clocks skew.

All six required integration tests pass (InitialElectionA, ReElectionA, ManyElectionA, LeaderStepsDownOn-HigherTermA, RapidLeaderChurnA, ExampleTestA).

2 Design

2.1 Raft State Machine

We implement the full Raft state machine from the extended Raft paper [1]. Each node is always in one of three roles: FOLLOWER, CANDIDATE, or LEADER. Role transitions are:

- **Follower** → **Candidate**: election timer fires without receiving a heartbeat or granting a vote within the randomized `election_timeout_ms` window (250–500 ms, seeded per-node to break symmetry).
- **Candidate** → **Leader**: majority of peers grant the vote in the same term.
- **Leader/Candidate** → **Follower**: any incoming RPC carries a term $>$ current term.

All Raft state (`current_term`, `voted_for`, `log`, `commit_index`, `last_applied`, `next_index`, `match_index`) is protected by a single mutex `mtx`. RPC handlers hold this mutex for their entire execution. Background threads release it before making outgoing gRPC calls, preventing a stalled peer from blocking the local server.

2.2 RPC Protocol

We define two RPCs in `proto/raft.proto`: `RequestVote` (`candidate` → `peers`) and `AppendEntries` (`leader` → `followers`, also used as heartbeats when `entries` is empty). Both request and reply messages carry a `lamport_ts` field that threads the Lamport clock through the network.

`AppendEntriesReply` also carries `conflict_term` and `conflict_index` to support the fast-backup optimization (Section 3.3).

2.3 RaftScope Instrumentation Layer

The `RaftEventLogger` class (205 lines in `inc/rafty/raft_scope.hpp`) attaches to each Raft node and writes newline-delimited JSON (JSONL) events to `logs/raft_scope_node_{id}.jsonl`. Every event record contains:

- `lamport_ts`: the Lamport clock value at event time
- `wall_time_us`: wall-clock microseconds (for layout only)
- `node_id, from_node, to_node`: causal endpoints
- `event_type`: one of 15 named types (`vote_request`, `vote_granted`, `log_append`, `commit`, `leader_change`, `node_crash`, `partition_start/end`, ...)
- `term, log_index, commit_index, role`: snapshot of consensus state

The logger owns a `LamportClock`: `tick()` on `send` (increments by 1), `update(received)` on `receive` (sets to `max(local, received) + 1` via a compare-exchange loop for thread safety). Heartbeat `AppendEntries` advance the clock but do not write an event record, keeping logs focused on meaningful state changes.

2.4 Space-Time Visualizer

The visualizer (`visualizer/index.html`) is a single self-contained HTML file using D3.js v7. It accepts multi-file JSONL drag-and-drop (or file picker) and renders:

- A vertical timeline per node, with events placed by their Lamport timestamp on the *y*-axis.
- Causal arrows from `from_node` at the send event to `to_node` at the matched receive event, colored by event type.
- A click-to-expand event detail panel showing all fields.
- Checkboxes to filter event types (e.g., hide heartbeats, show only commit/leader-change events).
- Live auto-refresh mode: polls the log files every 2 seconds and animates newly arrived events with a CSS “pop-in” keyframe, enabling real-time observation of a running cluster.

3 Implementation

3.1 Background Threads

Three detached threads are started by `Raft::run()`:

election_thread Sleeps in 10 ms increments. If elapsed time since `last_heartbeat_` exceeds `election_timeout_ms_` and the node is not a leader, it atomically increments the term, sets role to `CANDIDATE`, and fires a parallel wave of `RequestVote` RPCs (one thread per peer, each with a 200 ms deadline).

heartbeat_thread Waits on `heartbeat_cv_` with a 50 ms timeout. When the node is leader it spawns one thread per peer to call `send_append_entries_to`. The `propose()` function notifies this condvar to trigger immediate replication without waiting the full interval.

apply_thread Waits on `apply_cv_` until `last_applied_ < commit_index_`. Drains the committed suffix into a local vector under the lock, then calls `apply()` outside the lock to avoid deadlock with the framework’s message-queue.

3.2 Log Replication and Consistency Check

`AppendEntries` performs the standard two-phase log consistency check: it rejects the request if `prev_log_index` is beyond the local log length, or if `log[prev_log_index].term` does not match `prev_log_term`. When appending, it truncates any conflicting suffix before inserting new entries, preserving the log matching property.

Commit advancement uses the standard majority rule: the leader scans backwards from the last log index looking for the highest N such that `log[N].term == current_term` and a majority of `match_index[i] >= N`.

3.3 Fast-Backup Optimization

Naive log replication decrements `next_index` by one per rejected `AppendEntries`, requiring $O(\text{stale_entries})$ round trips after a partition heals. We implement the fast-backup protocol: on rejection the follower returns the term of the conflicting entry (`conflict_term`) and the first index of that term (`conflict_index`). The leader searches its log for the last entry with `conflict_term`; if found it sets `next_index` to just past it, otherwise it uses `conflict_index` directly. This lets a rejoining node catch up in a single round trip per stale term rather than per stale entry.

3.4 Safety Properties Maintained

- **Election safety:** a node only votes once per term (`voted_for` persists until a term change) and only for candidates whose log is at least as up-to-date as its own (comparison by `last_log_term` then `last_log_index`).
- **Log matching:** enforced by the consistency check and truncation-before-append logic.
- **Leader completeness:** the log up-to-date check in `RequestVote` prevents a candidate missing committed entries from winning an election.
- **Granting a vote resets the election timer:** prevents a freshly voted-for candidate from being immediately challenged.

4 Evaluation

4.1 Correctness: Integration Tests

All six required integration tests pass consistently:

Test	Nodes	What it checks
InitialElectionA	3	One leader elected; term stable without failures
ReElectionA	3	New leader after leader disconnected; old leader rejoins
ManyElectionA	7	Correct election under repeated random disconnections
LeaderStepsDownOnHigherTermA	3	Leader reverts to follower on seeing higher term
RapidLeaderChurnA	5	Liveness under rapid, repeated leader kills
ExampleTestA	3	End-to-end propose + apply

The election timeout range of 250–500 ms with a 50 ms heartbeat gives a timeout-to-heartbeat ratio of 5–10 $\times$, well above the recommended lower bound to avoid false timeouts on a loaded test machine.

4.2 Observability: Space-Time Diagram

To exercise the visualizer we ran a 5-node cluster under `scripts/run_fault_injection.sh`, which injects random kills and network partitions over 60 seconds. The resulting JSONL logs (one per node) were dropped into the visualizer. We confirmed:

1. Causal arrows are consistent: every `vote_request` arrow terminates at the corresponding `vote_`

- `recv` on the peer, at a Lamport time strictly greater than the send.
2. Leader-change events cluster at term boundaries, making split-vote rounds visually obvious.
 3. Commit events on the leader appear after the corresponding `log_append` arrows reach a majority of followers, verifiable by inspection.
 4. The live-refresh mode correctly streams new events into the diagram without re-rendering existing structure, confirmed by a 30-second real-time observation session.

4.3 Performance

We measured round-trip latency for log replication on a single machine (loopback gRPC) at approximately 1–3 ms per entry. The 50 ms heartbeat interval means commits typically land within one heartbeat interval of a `propose()` call. The fast-backup optimization was validated by the `BackupExtraB` test (which requires catching up a follower with hundreds of stale entries in ≤ 30 RPC round trips).

5 Conclusion

RaftScope demonstrates that embedding causal clocks directly into the consensus protocol (at the RPC field level) adds minimal overhead (one 64-bit integer per message) while enabling rich post-hoc and real-time debugging through a browser-based visualizer. The implementation passes all required correctness tests and the instrumentation layer does not interfere with timing-sensitive election behavior.

References

- [1] Diego Ongaro and John Ousterhout. *In Search of an Understandable Consensus Algorithm (Extended Version)*. USENIX ATC 2014.

AI Usage Report

Overview

We used **Claude Code** (Anthropic’s CLI agent, powered by the `claude-sonnet-4-6` model) to build the RaftScope visualizer (`visualizer/index.html`). Claude Code differs from a chat assistant in that it can read, write, and execute code directly in the project directory and observe the output, behaving more like a pair-programmer with shell access than a question-answering tool.

What the AI Built

We described the desired space-time layout (nodes as columns, Lamport time as vertical axis, causal arrows between matching send/receive events) and Claude Code produced the full implementation: the D3.js v7 rendering loop, the multi-file JSONL drag-and-drop parser, the event-type color map, the click-to-expand sidebar panel, and the checkbox filter controls. The live auto-refresh mode (polling log files every 2 seconds and animating newly arrived events with a CSS `pop-in` keyframe) was also AI-generated in a single prompt round.

What Went Well

Speed of scaffolding. The D3 boilerplate, SVG coordinate math, and file-drop handler would have taken several hours to write by hand. Claude Code produced a working first draft in under ten minutes, letting us focus on what the diagram should *show* rather than how to wire up the DOM.

Incremental feature additions. Adding the live-refresh mode, the event-detail sidebar, and the arrow-label overlays each required a single focused prompt. The AI preserved the existing structure and spliced in only the new logic, which made reviewing diffs straightforward.

What Did Not Work Well

Visual layout tuning. The AI cannot see the rendered result. Iterating on spacing, font sizes, arrow curvature, and legend placement required us to open the browser, observe what looked wrong, describe it in text, and repeat. Several rounds were needed for the causal arrow routing in particular: the first draft drew arrows that crossed node timelines in confusing ways.

Edge-case JSONL handling. The initial parser crashed silently on malformed or truncated lines (which occur when a node is killed mid-write). We had to explicitly describe the failure mode before the AI added the `try/catch` guard around each `JSON.parse` call.

Overall Assessment

Claude Code was the right tool for the visualizer. The component is primarily JavaScript and CSS with a clear, self-contained spec, so the AI could generate correct code quickly and iterative refinement was cheap. The main limitation (not being able to see the rendered output) slowed layout work, but for logic and structure the collaboration was efficient.

Appendix: Representative Prompts

Prompt 1 (initial visualizer):

“Build a single self-contained HTML file using D3.js v7. It accepts a drag-and-drop of multiple JSONL files (one per Raft node). Each line is a JSON object with fields: `lamport_ts`, `node_id`, `event_type`, `from_node`, `to_node`, `term`, `log_index`, `commit_index`, `role`. Render a space-time diagram: one vertical timeline per node (x-axis), events placed by `lamport_ts` on the y-axis as colored circles. Draw causal arrows from the send event on `from_node` to the matching receive event on `to_node`. Include a sidebar that shows all fields for the clicked event.”

Prompt 2 (live auto-refresh):

“Add a live auto-refresh mode to the visualizer. Every 2 seconds, re-read the JSONL files (same paths as last load). Only render events with `lambport-ts` greater than the last-seen max. Animate new events with a pop-in CSS keyframe. Show a green pulsing dot in the header when live mode is on. Don’t re-render the entire SVG; only append new circles and arrows.”

Prompt 3 (crash resilience):

“The parser crashes when a JSONL file has a truncated last line, which happens when a node is killed mid-write. Wrap each `JSON.parse` in a try/catch and skip malformed lines silently. Also add a small toast notification showing how many lines were skipped on the last load.”